



DÍA 2 • MARTES 08:00 - 13:00

Práctica en Kabré: de los datos al modelo

Los notebooks 01 y 02 paso a paso, con sus retos

El plan de hoy

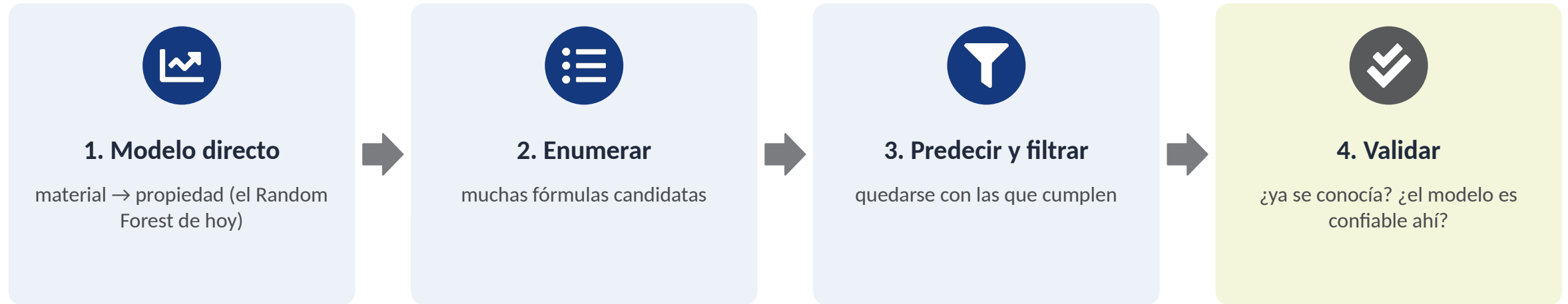
Un minuto de teoría pendiente y el resto es práctica

Hora	Bloque	Actividad
08:00	T2.4	Diseño inverso en un minuto (pendiente del lunes)
08:05	Kabré	Entrar a OnDemand y abrir el curso (para quien no lo hizo en casa)
08:25	Notebook 01	Celdas [1] a [22]: datos, estructuras y descriptores Magpie. Retos 1 y 2
09:50		Receso
10:20	Notebook 02	Celdas [1] a [18]: Random Forest, Materials Project, screening y SHAP. Retos 1 y 2
12:15	Retos	Tiempo para terminar los retos de cualquiera de los dos notebooks
12:45		Cierre y ejercicio para casa

Términos **Diseño inverso:** partir de la propiedad deseada y buscar el material · **Kabré:** supercomputadora del CNCA · **OnDemand:** portal web de Kabré; abre Jupyter en el navegador · **Notebook:** documento con celdas de código y texto que se ejecutan en orden · **Celda:** en un cristal, la celda unitaria: bloque mínimo que se repite (a, b, c, α , β , γ); en un notebook, un bloque de código · **Descriptor (feature):** número que describe un material y que el modelo usa como entrada · **Magpie:** 132 descriptores de composición: 22 propiedades $\times$ 6 estadísticas · **Random Forest:** promedio de muchos árboles de decisión entrenados al azar · **Materials Project:** base abierta de más de 150 000 materiales calculados con DFT · **Screening:** evaluar con un modelo rápido muchos candidatos y quedarse con los mejores · **SHAP:** reparte cada predicción entre los descriptores: valor base + aportes = predicción

Diseño inverso en un minuto

De "¿qué propiedad tiene este material?" a "¿qué material tiene esta propiedad?"



- Hoy lo hacen con sus manos en el notebook 02 (celdas [12] a [14]): 270 fórmulas, 13 con $K > 300$ GPa.
- Un candidato del modelo es una hipótesis para DFT o experimento, no un descubrimiento. El miércoles generamos candidatos y el jueves optimizamos.

Términos Validación: comprobar el modelo con datos que no vio • **GPa:** gigapascal, unidad de presión y de módulo elástico • **DFT:** teoría del funcional de la densidad: cálculo cuántico de energía y electrones

Paso 0: entrar a Kabré y abrir el curso

Nada que instalar: todo corre en la supercomputadora del CNCA desde el navegador



¿Todavía no tiene `~/curso`? en una terminal: `cp /work/trojas/ia_cm_cnca/ia_cm_curso_v3_kabre.zip ~ && cd ~ && unzip -o ia_cm_curso_v3_kabre.zip`

Si Jupyter pregunta el kernel: elija Python-AI-Materials, no Python 3.

Términos **CNCA**: Colaboratorio Nacional de Computación Avanzada, del CeNAT · **Jupyter**: entorno web para trabajar con notebooks · **kura**: cola de CPU de Kabré donde corre Jupyter · **Kernel**: en Jupyter, el proceso de Python que ejecuta las celdas; en un GP, la función que mide el parecido entre dos puntos · **Python**: lenguaje de programación usado en el curso

Cómo leer un notebook del curso



Celdas en orden

Shift + Enter, de arriba hacia abajo. Los números [n] de la izquierda son los que usamos en estas láminas.



"Cómo funciona"

Recuadros azules que explican las líneas menos obvias. Léalos antes de modificar el código.



Retos

Recuadros naranja con una celda vacía: ahí escribe usted. No hay una única respuesta.



Si se reinicia el kernel

Hay que volver a correr desde arriba: las variables se pierden. Si un visor no aparece, recargue la página (F5) una vez.



NOTEBOOK 01 · 85 min

Exploración de datos de materiales

Celdas [1] a [22] · datos, estructuras y descriptores



CeNAT
Centro Nacional de Alta Tecnología

01 • Paso 1: preparar el entorno

Celdas [1] a [3]

```
import pandas as pd, pymatgen.core, matminer, sklearn
print(f"matminer {matminer.__version__}")
DATOS = encontrar_datos() # busca la carpeta datos/
```

[1]: estilo de los gráficos y visores (plotly, 3D).

[2]: importa las librerías y muestra sus versiones.

[3]: ubica la carpeta datos/ del curso.

Deben ver: versiones (pymatgen 2024.10.29, matminer 0.9.3, sklearn 1.7.2) y ningún error.

Si algo falla aquí, no avance: casi siempre es el kernel equivocado.

Términos **pandas:** biblioteca de Python para tablas de datos • **pymatgen:** biblioteca de Python para estructuras cristalinas • **matminer:** biblioteca de Python con datasets y descriptores de materiales • **scikit-learn:** biblioteca de Python de modelos clásicos de ML

01 • Paso 2: cargar e inspeccionar el dataset

Celdas [4] a [6]

```
from matminer.datasets import load_dataset
df = load_dataset("elastic_tensor_2015")
df.head(3)
df.isnull().sum()
df[["K_VRH", "G_VRH"]].describe()
```

Qué es: propiedades elásticas calculadas con DFT para 1181 cristales (de Jong 2015, Materials Project). Viene en datos/, no se descarga.

Deben ver: 1181 filas y 20 columnas, ningún nulo; K promedio 136 GPa y G 68 GPa.

Las tres preguntas ante cualquier dataset: ¿qué contiene?, ¿hay huecos?, ¿en qué escala están los números?

Columnas clave: formula, structure (objeto de pymatgen), K_VRH, G_VRH, space_group.

Términos **Módulo volumétrico K:** resistencia a cambiar de volumen bajo presión uniforme · **Módulo de corte G:** resistencia a cambiar de forma sin cambiar de volumen · **Grupo espacial:** una de las 230 combinaciones de simetría de un cristal

Las columnas del dataset en palabras sencillas

Columna	Qué mide, en sencillo	Ejemplos del dataset
K_VRH (GPa)	Qué tanto se resiste a que lo compriman por todos lados, como apretar una pelota con las dos manos. Más alto = más difícil de comprimir.	Diamante 436 · Fe 182 · Al 83 · Pb 37 (promedio 136)
G_VRH (GPa)	Qué tanto se resiste a deformarse de lado, como empujar la tapa de un libro sin mover la base. Va de la mano con la dureza.	Diamante 523 · W 147 · Al 24 · Pb 13 (promedio 68)
VRH	Un cristal no responde igual en todas las direcciones. Voigt y Reuss son el máximo y el mínimo posibles; VRH es su promedio: el valor típico de un pedazo con muchos granos.	K_Voigt, K_Reuss, G_Voigt y G_Reuss también vienen en la tabla
elastic_anisotropy	Qué tan distinto responde según la dirección. 0 = igual en todas; un número grande = muy distinto, como un material en capas.	W 0.01 · Al 0.65 · Cu 8.9 · carbono en capas hasta 397 (mediana 0.36)
poisson_ratio	Al estirarlo, cuánto se adelgaza. Corcho $\approx$ 0, metales $\approx$ 0.3, caucho $\approx$ 0.5.	Be 0.04 · diamante 0.07 · Fe 0.32 · AlV ₃ 0.47
nsites	Cuántos átomos hay en la caja mínima que se repite para formar el cristal (la celda). No es el tamaño del material: es qué tan compleja es su estructura.	SiOs 2 · Nb ₈ Co ₂ Si ₂ 12 · Li ₁₂ Si ₇ 152 (mediana 10)
volume (Å ³)	El volumen de esa caja. Más átomos en la caja = más volumen (correlación 0.94).	Be 16 · Li ₁₂ Si ₇ 2399 (mediana 169)
space_group	Un número de 1 a 230 que resume la simetría del cristal. De él sale el sistema cristalino (celda [8]).	225 = Ir (cúbico) · 194 = Os (hexagonal)

Idea clave 1 GPa $\approx$ 10 000 atmósferas. K y G son las dos que vamos a mirar y predecir.

Términos **Promedio VRH:** media de las cotas de Voigt (techo) y Reuss (piso) para un policristal · **Coefficiente de Poisson:** contracción lateral entre alargamiento · **Correlación:** qué tanto dos variables cambian juntas (de -1 a 1); no implica causa · **Sistema cristalino:** 7 familias según la forma de la celda (de triclinico a cúbico)

01 • Paso 3: cuatro miradas a los datos

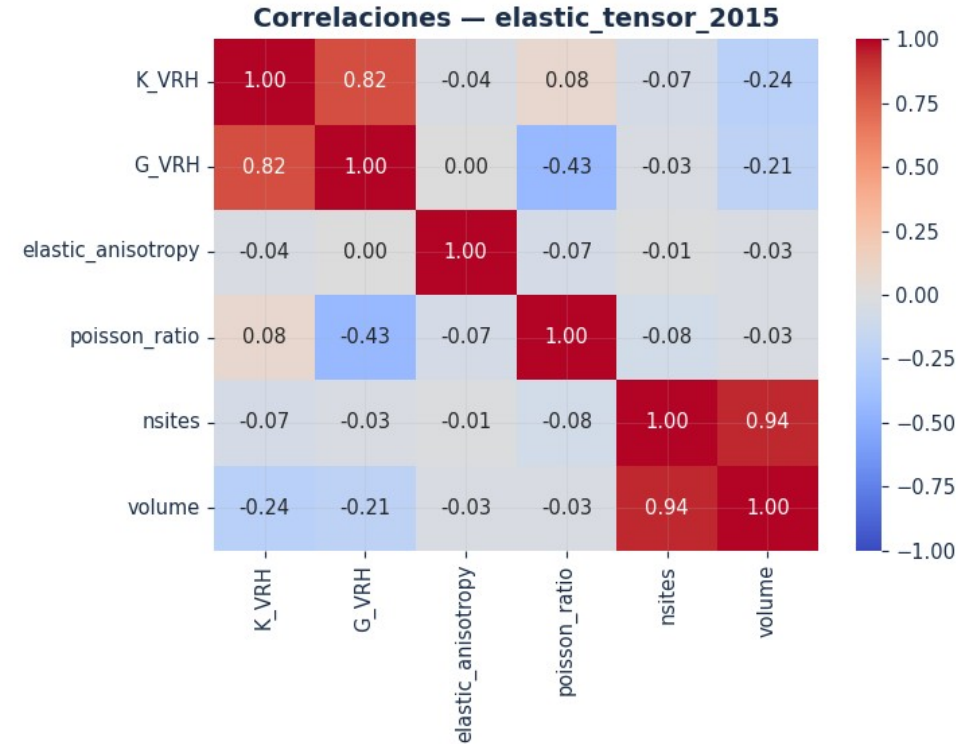
Celdas [7] a [10]

[7] Histogramas: ¿por qué la anisotropía tiene una cola tan larga? Unos pocos materiales en capas (el grafito llega a 397).

[8] Sistemas cristalinos: se deducen del número de grupo espacial. 452 cúbicos y ningún triclinico: el dataset refleja lo que alguien decidió calcular.

[9] Correlaciones: K y G se mueven juntos (0.82) porque los dos dependen de la fuerza del enlace.

[10] K contra G: en Kabré es interactivo; el punto de arriba a la derecha es el diamante (G = 523 GPa).

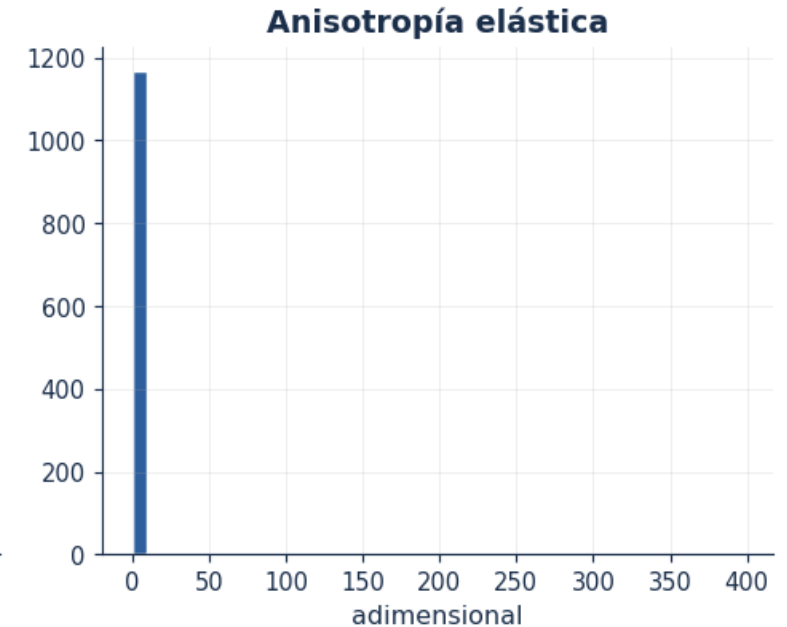
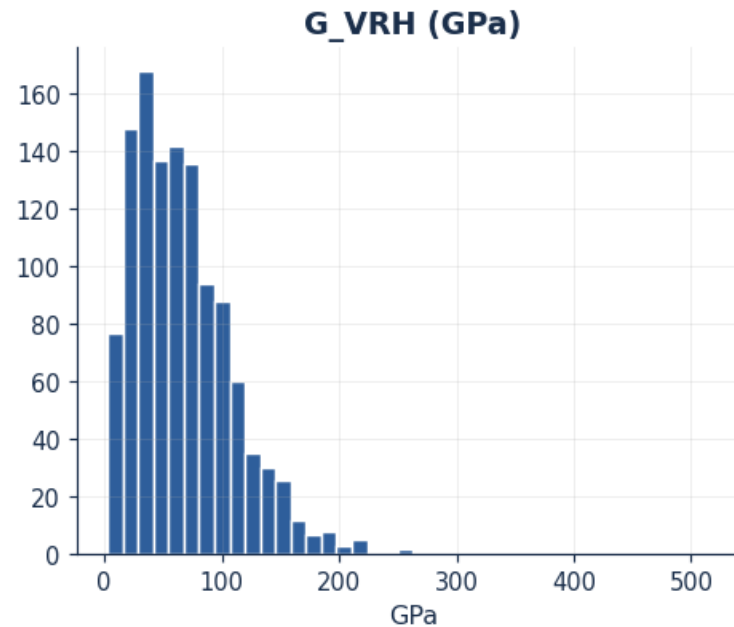
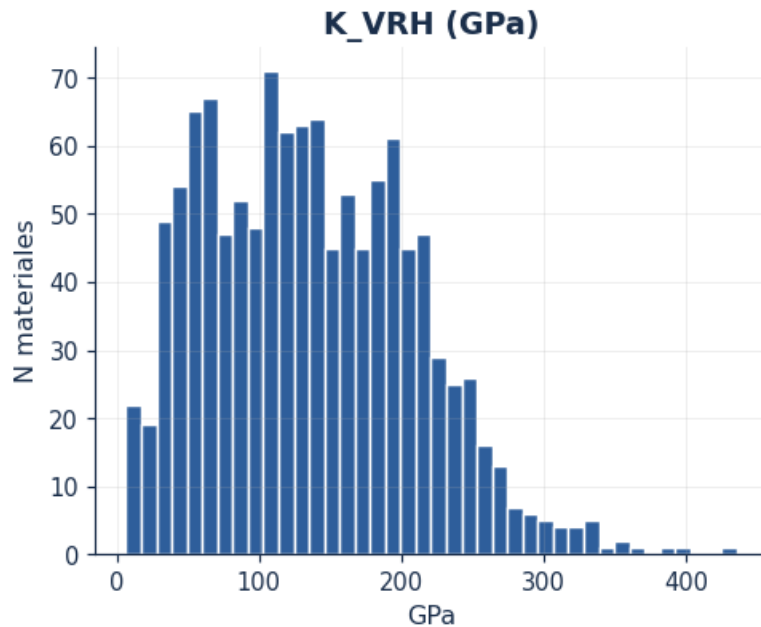


Términos **Anisotropía:** propiedades distintas según la dirección; **isótropo:** iguales

01 • Paso 3 (cont.): lo que deben ver

Celdas [7] y [8]

- Distribuciones sesgadas a la derecha: muchos materiales blandos y pocos muy rígidos. El Random Forest del notebook 02 va a fallar más justo en esa cola.



Reto 1 del notebook 01 (5 min)



Celda [11]

- ¿Qué sistema cristalino tiene el mayor K_{VRH} promedio?
- ¿Cuál es el más anisótropo en promedio?
- ¿Es el mismo? ¿Tiene sentido físico?



Pista única

- `groupby: df.groupby("sistema")[...].mean()`
- Compare también la mediana: la anisotropía tiene una cola enorme

01 • Paso 4: una estructura por dentro con pymatgen

Celdas [12] y [13]

```
struct = df["structure"].iloc[0]
struct.formula, struct.density
struct.get_space_group_info()
struct.get_neighbors(struct[0], r=3.0)
```

Structure: red + posiciones + especies en un solo objeto.

Deben ver: Nb₈Co₂Si₂, tetragonal (P4/mcc, 124), 12 átomos, 7.8 g/cm³.

Vecinos: el Nb tiene 7 a menos de 3 Å; el más cercano es Co a 2.60 Å.

Ojo: el cristal es periódico; un vecino puede estar en la celda de al lado.

01 • Paso 5: ver estructuras y su composición

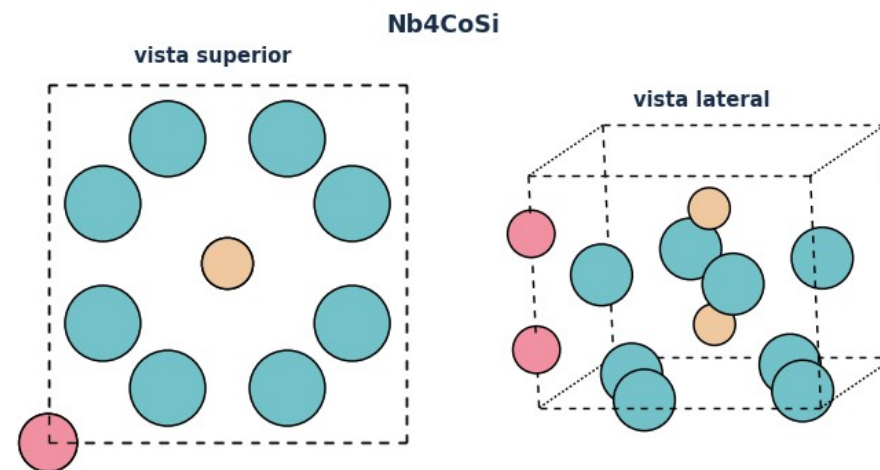
Celdas [14] a [17]

```
ver_estructura(struct)          # visor 3D, se gira con el mouse
comp = Composition(df["formula"].iloc[5])
comp.fractional_composition
Element("Fe").X                  # electronegatividad
```

[14] a [16]: $\text{Nb}_8\text{Co}_2\text{Si}_2$ y Be_8Re_4 (hexagonal) en 3D. Compare dos sistemas cristalinos.

[17]: AlCo_3C tiene fracciones 0.2, 0.6 y 0.2; el Fe tiene electronegatividad 1.83.

Por qué importa: fracciones y propiedades de los elementos son los ladrillos de los descriptores del paso 6.



Términos **Electronegatividad:** tendencia de un átomo a atraer electrones en un enlace

Reto 2 del notebook 01 (10 min)



Celda [18]

- Encuentre el material más denso de todo el dataset
- Visualícelo en 3D con su celda unitaria



Sin pistas de código

- Ya usaron todo lo necesario: `.density` de cada `structure` y `ver_estructura`
- Verificación: la densidad ganadora supera 20 g/cm^3

01 · Paso 6: de la fórmula a 132 descriptores Magpie

Celdas [19] y [20]

```
stc = StrToComposition(target_col_id="composition")
df = stc.featurize_dataframe(df, "formula")
ep = ElementProperty.from_preset("magpie")
df = ep.featurize_dataframe(df, col_id="composition")
```

El paso más importante del día: un modelo no lee "Fe₂O₃", necesita números.

Magpie: 22 propiedades de los elementos × 6 estadísticas (mínimo, máximo, rango, media, desviación, moda) = 132.

Ejemplo: electronegatividad media de Fe₂O₃ = $0.4 \times 1.83 + 0.6 \times 3.44 \approx 2.80$.

Deben ver: 132 descriptores; la tabla pasa de 20 a 154 columnas.

01 • Paso 7: ¿qué descriptores "ven" la física? y guardar

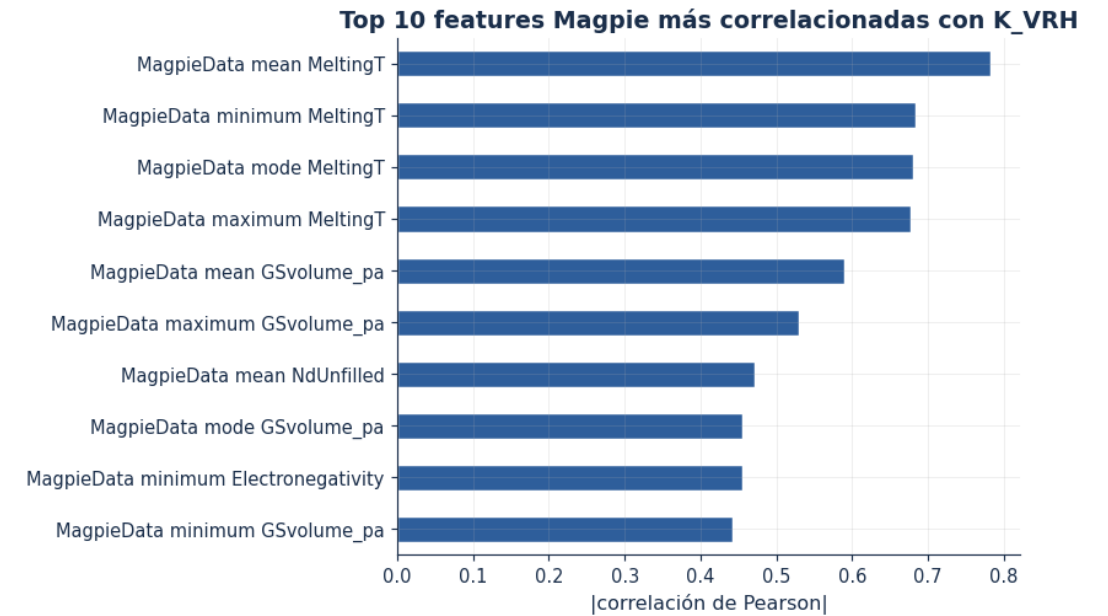
Celdas [21] y [22]

```
corr = df[feature_cols + ["K_VRH"]].corr()["K_VRH"]  
df.to_pickle(DATOS / "df_con_features_dia1.pkl")
```

[21]: la temperatura de fusión media correlaciona 0.78 con K.

Pregunte: ¿por qué un material que funde alto tiende a ser rígido? Ambos dependen de la fuerza del enlace: correlación, no causa.

[22]: guarda el archivo que el notebook 02 necesita. Sin él, el 02 no arranca.



Ejercicio del notebook 01 para casa (opcional)

- Los 5 materiales con mayor elastic_anisotropy y sus grupos espaciales.
- Agregue el featurizer Meredig de matminer: ¿cuántos descriptores genera? ¿Cuál correlaciona más con G_VRH?
- Calcule la densidad de los 10 primeros materiales con pymatgen (struct.density) en una columna nueva. ¿Correlaciona con K_VRH?

Idea clave Antes del receso: cada persona debe tener guardado datos/df_con_features_dia1.pkl.

Términos **Featurización**: convertir una fórmula o estructura en un vector de descriptores



NOTEBOOK 02 · 115 min

Predicción de propiedades y diseño inverso

Celdas [1] a [18] · Random Forest, Materials Project, screening y SHAP

02 • Paso 1: cargar lo que hicieron en el 01

Celdas [1] y [2]

```
df = pd.read_pickle(DATOS / "df_con_features_dia1.pkl")  
print(df.shape)    # (1181, 154)
```

Deben ver: Dataset del Día 1: (1181, 154).

FileNotFoundError: no corrieron el 01 hasta la celda [22]. Vuelvan al 01, córranlo completo y regresen.

Por qué: la featurización ya se hizo una vez; no se repite.

02 • Paso 2: X, y, separar y escalar

Celdas [3] y [4]

```
X = df[feature_cols].dropna()          # 132 descriptores
y = df.loc[X.index, "K_VRH"]          # lo que queremos predecir
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)  # aprende
X_test_sc = scaler.transform(X_test)       # solo aplica
```

La regla de oro: el modelo se evalúa solo con datos que nunca vio.

Deben ver: 944 materiales para entrenar y 237 para evaluar.

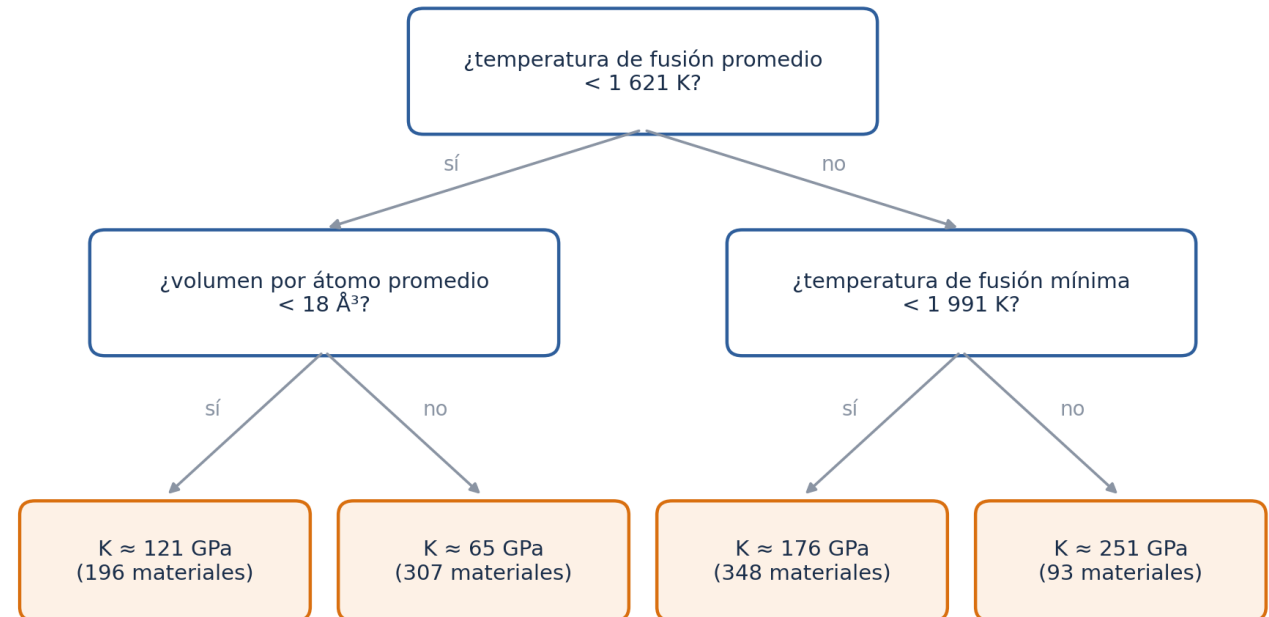
fit solo en train: si el escalador viera el test, el error parecería menor (fuga de información).

random_state=42: todos obtienen lo mismo.

Términos **Entrenamiento y prueba:** datos con los que el modelo aprende y datos reservados para evaluarlo • **Semilla aleatoria:** número que fija el azar para poder repetir un resultado • **Escalado:** llevar cada descriptor a media 0 y desviación 1 (StandardScaler) • **Fuga de información:** datos de prueba que se cueban en el entrenamiento e inflan las métricas

Random Forest (1/5): un árbol de decisión

- Un árbol adivina K como en el juego de las 20 preguntas: hace preguntas de sí o no sobre los descriptores.
- Cada pregunta separa los materiales en dos grupos lo más parecidos posible por dentro.
- Al final de cada camino hay una hoja: la predicción es el K promedio de los materiales de entrenamiento que llegaron ahí.
- Este es un árbol real de 2 niveles con nuestros datos. El de la celda [5] hace cientos de preguntas.



Cada hoja predice el K promedio de los materiales de entrenamiento que llegaron ahí

Idea clave El árbol no sabe física: solo aprende qué preguntas separan bien a los materiales.

Términos **Árbol de decisión**: modelo que divide los datos con preguntas del tipo $\{x > \text{umbral}\}$

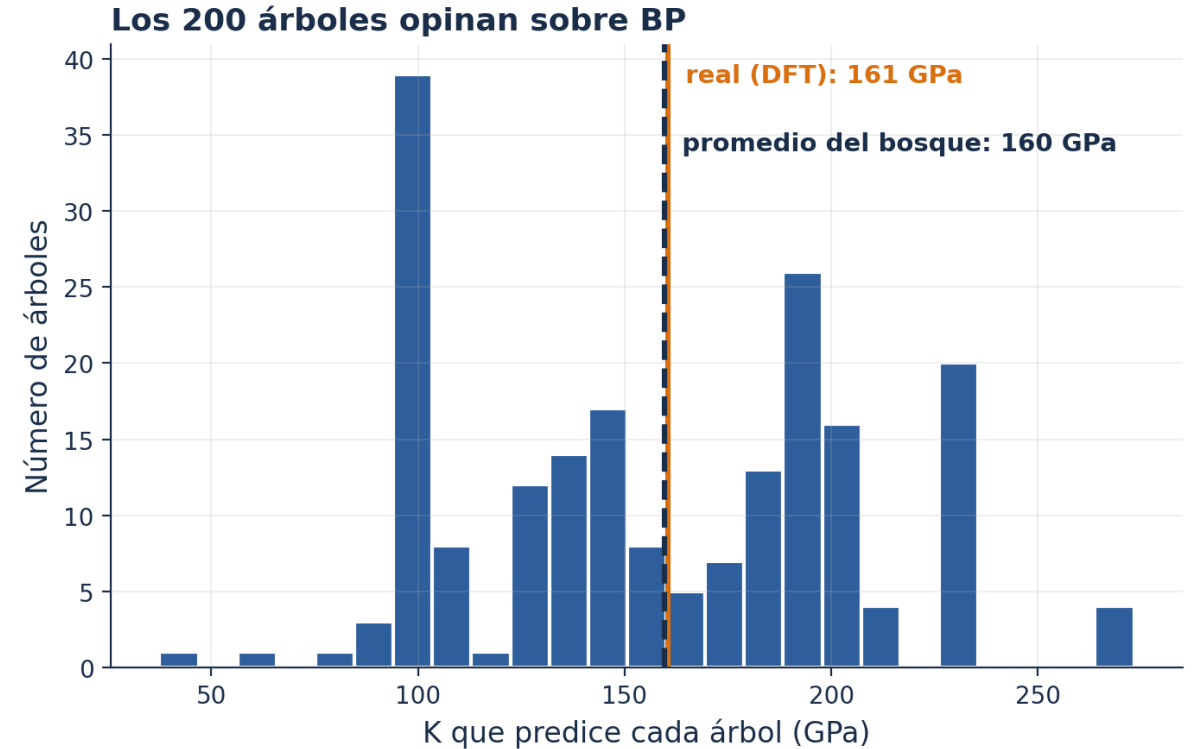
Random Forest (2/5): un bosque que opina y se promedia

- Un solo árbol se equivoca mucho: aprende de memoria los detalles de sus datos.

La idea: entrenar 200 árboles distintos. Cada uno ve una muestra al azar de los materiales y, en cada pregunta, un grupo al azar de descriptores.

La predicción: el promedio de los 200. Los errores de unos se compensan con los de otros.

En la figura: para el BP los árboles dicen de 40 a 270 GPa; el promedio da 160 y el valor real es 161.



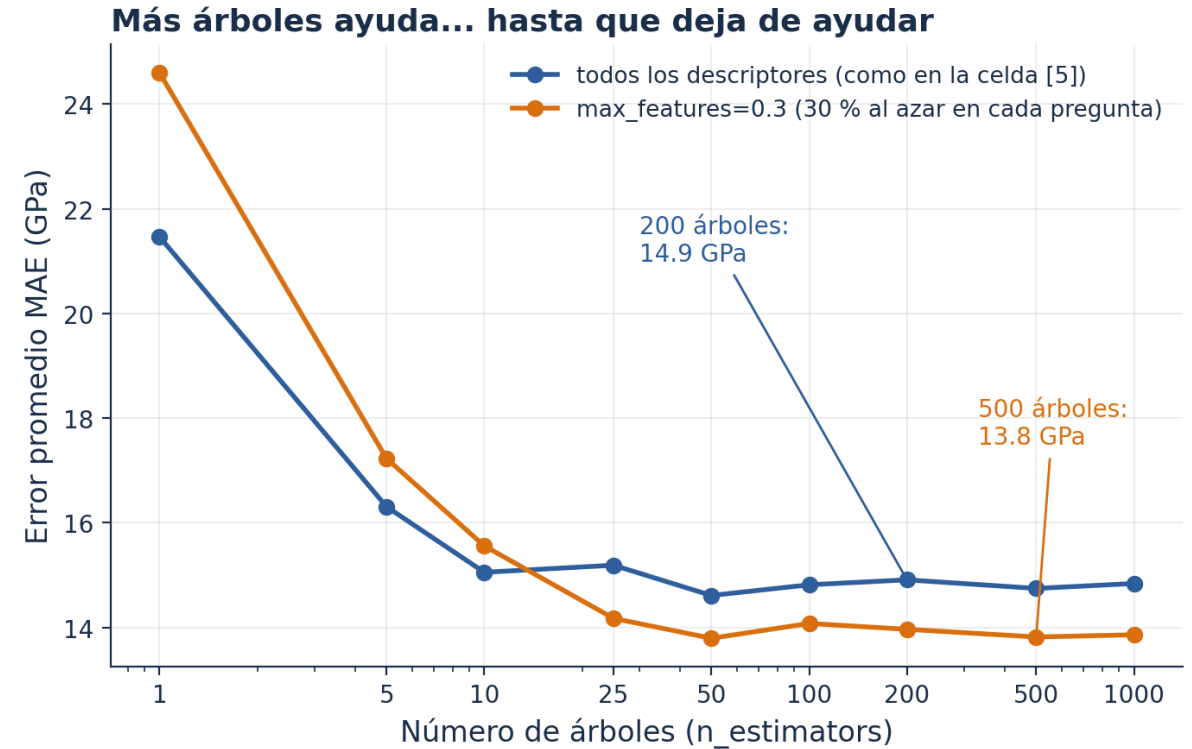
Random Forest (3/5): las perillas del bosque → Reto 1b

n_estimators: cuántos árboles. Más ayuda al principio; después de unos 50 a 500 ya casi no cambia.

max_features: qué fracción de descriptores mira cada pregunta. Con 0.3 los árboles salen más distintos entre sí y el promedio mejora.

random_state: fija el azar para que todos obtengan lo mismo.

Reto 1b, celda [9]: copiar la celda [5] como rf2, cambiar estas perillas y ver cuánto baja el error (de 14.9 a 13.8 GPa).



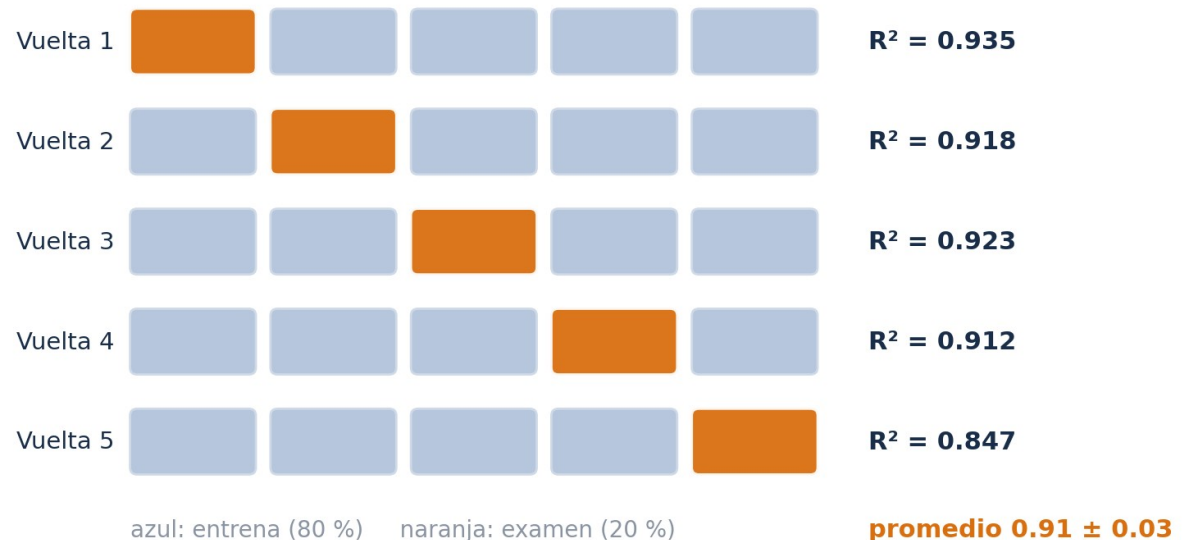
Random Forest (4/5): ¿fue suerte? → Reto 1a

- En la celda [4] separamos una sola vez: 80 % para entrenar y 20 % para el examen. ¿Y si nos tocó un examen fácil?

Validación cruzada: repetir el examen 5 veces, cada vez con otro 20 %, y mirar los 5 resultados.

Resultado: R^2 de 0.85 a 0.94, promedio 0.91 ± 0.03 . El 0.904 de la celda [5] no fue suerte.

Reto 1a, celda [8]: una línea con `cross_val_score` hace las 5 vueltas.



Términos **Validación cruzada:** entrenar k veces dejando cada parte de los datos una vez fuera · R^2 : fracción de la variación que explica el modelo; 1 es perfecto

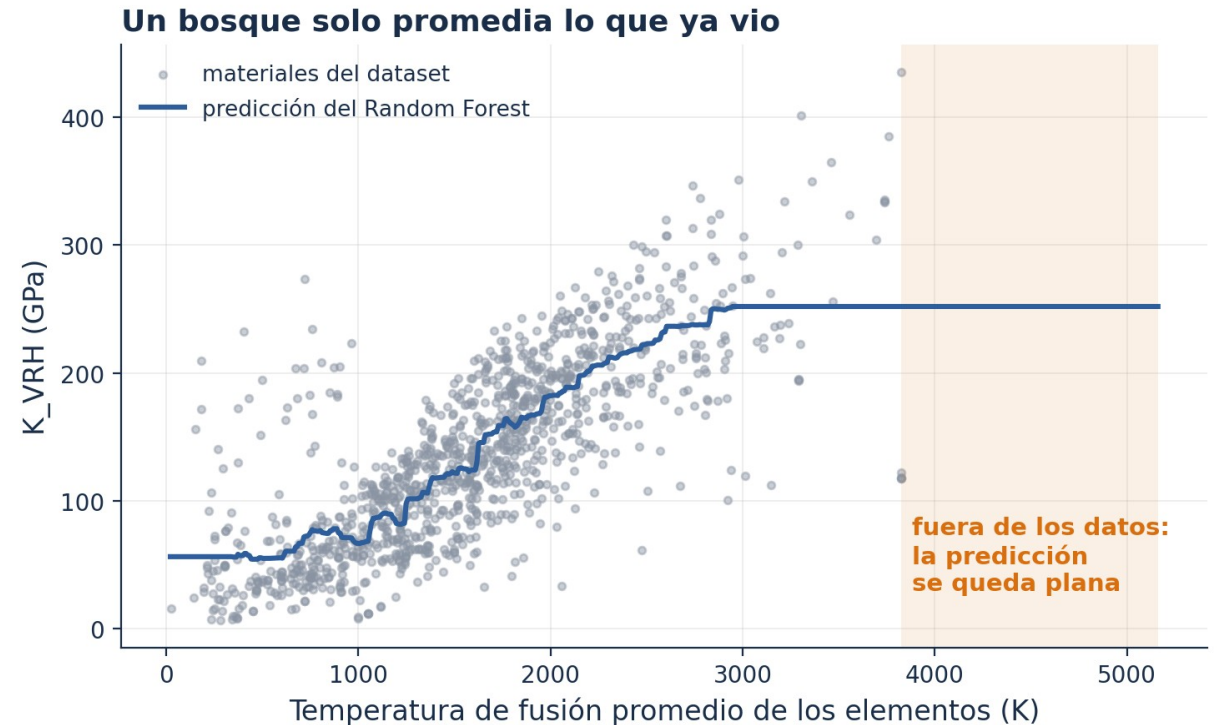
Random Forest (5/5): lo que no puede hacer → Reto 2

- Cada hoja predice un promedio de materiales que ya vio. Por eso el bosque nunca predice algo más rígido que lo más rígido que conoce.

En la figura: dentro de los datos sigue la tendencia; más allá, la predicción se queda plana.

En el notebook: WC real 385 GPa, predicho 321; ningún candidato del screening pasa de 332 GPa.

Reto 2, celda [15]: con ternarios el máximo es 318 GPa. ¿Confiamos en predicciones lejos de lo que vio?



Idea clave Un Random Forest interpola bien y no extrapola: para buscar algo nunca visto hace falta otra herramienta (miércoles).

Términos **Binario**, **ternario**, **cuaternario**: compuestos de 2, 3 o 4 elementos · **Extrapolar**: predecir fuera de la región de los datos de entrenamiento

02 • Paso 3: entrenar el Random Forest y medir

Celdas [5] y [6]

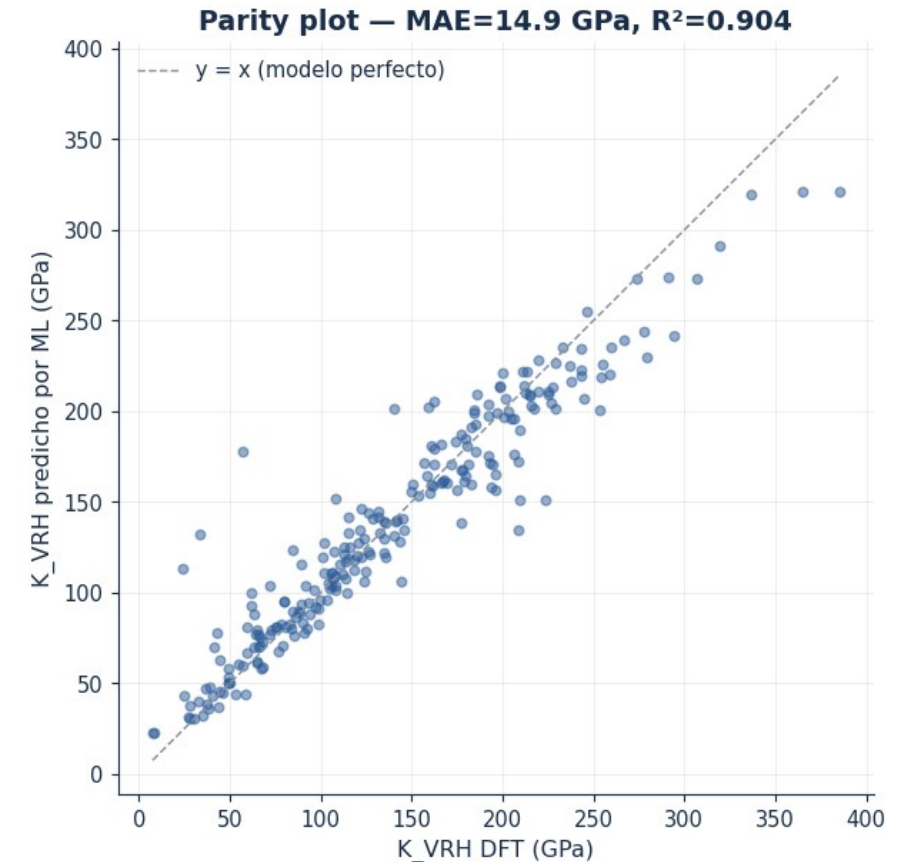
```
rf = RandomForestRegressor(n_estimators=200,  
                           random_state=42)  
rf.fit(X_train_sc, y_train)  
y_pred = rf.predict(X_test_sc)  
mean_absolute_error(y_test, y_pred)
```

Random Forest: 200 árboles, cada uno aprende de una muestra distinta; la predicción es el promedio.

Deben ver: MAE 14.9 GPa y R^2 0.904.

Comparar con algo: predecir siempre la media da un MAE de 60 GPa. El modelo es 4 veces mejor.

Parity plot [6]: ¿dónde se abre la nube? En los más rígidos: hay pocos datos ahí y el modelo no pasa de unos 330 GPa.



Términos **MAE:** error absoluto medio, en las unidades de la propiedad • **Pocos datos (small data):** decenas o pocos cientos de experimentos; piden modelos simples con incertidumbre

Todos los parámetros del entrenamiento en una tabla

Parámetro (celda)	Valor	Qué hace, en sencillo	Si lo cambia (error MAE en GPa; base 14.9)
test_size [4]	0.2	Guarda el 20 % para el examen final, sin que el modelo lo vea.	0.1 → 14.1 · 0.3 → 14.8. Con menos examen el número es menos confiable.
random_state [4] y [5]	42	Fija el azar para que todos obtengan lo mismo.	Otro reparto de los datos: entre 11.6 y 15.1. Por eso existe la validación cruzada.
StandardScaler [4]	fit en train	Pone los descriptores en la misma escala.	Sin escalar: 14.9, igual. Los árboles solo comparan mayor o menor. Sí importa en otros modelos.
n_estimators [5]	200	Cuántos árboles votan.	Más árboles = más estable y más lento. Después de unos 200 casi no mejora.
max_features (no aparece)	1.0 (todos)	Cuántos descriptores mira cada pregunta.	0.3 con 500 árboles → 13.8 (Reto 1b): árboles más distintos entre sí.
max_depth (no aparece)	sin límite	Cuántas preguntas seguidas hace cada árbol.	3 → 27.2 · 5 → 19.9 · 10 → 15.1. Muy corto = el árbol no alcanza a aprender.
min_samples_leaf (no aparece)	1	Mínimo de materiales en cada hoja.	5 → 16.1 · 30 → 23.1. Suaviza las predicciones; útil si los datos tienen ruido.
bootstrap (no aparece)	True	Cada árbol aprende de una muestra al azar.	False → 20.0: los 200 árboles salen iguales; el bosque vale lo que un árbol.
n_jobs [5]	-1	Núcleos en paralelo (-1 = todos).	No cambia el resultado, solo la velocidad.

Idea clave Cambiar solo el reparto de los datos mueve el error más que casi cualquier perilla: compare siempre con validación cruzada.

Términos **Bootstrap**: remuestrear los datos con reemplazo para estimar la variabilidad

02 • Paso 4: ¿qué descriptores usa el modelo?

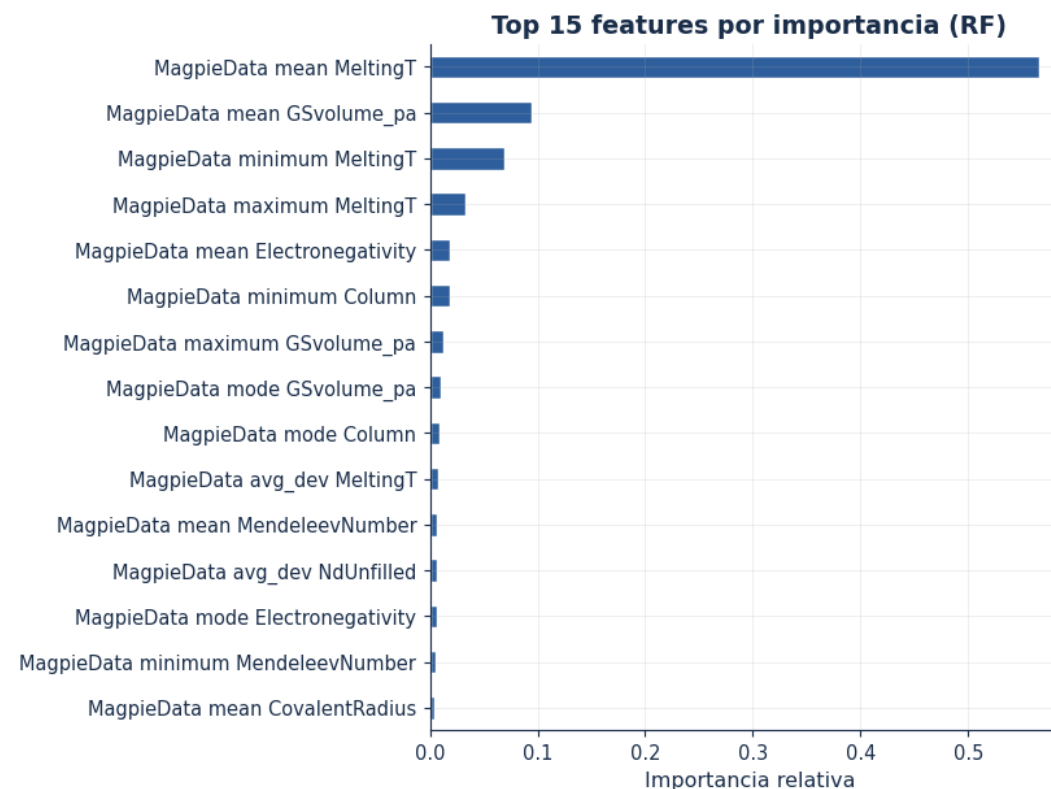
Celda [7]

```
pd.Series(rf.feature_importances_, index=feature_cols)
    .nlargest(15).plot(kind="barh")
```

Deben ver: la temperatura de fusión media domina, muy por encima del resto.

Lectura: es un indicador de la fuerza del enlace. El modelo "redescubrió" física.

Cuidado: esta importancia (por impureza) favorece variables con muchos valores distintos. SHAP, al final del notebook, es más fino.



Términos **Importancia por impureza:** cuánto reduce el error un descriptor en los cortes de los árboles

Reto 1 del notebook 02 (10 min)



Celdas [8] y [9]

- [8] R^2 con validación cruzada de 5 folds (cross_val_score ya está importado)
- [9] Baje el MAE por debajo de 14 GPa ajustando hiperparámetros. ¿Cuáles movió? ¿Cuándo dejó de mejorar?



Referencias

- Con 500 árboles y max_features=0.3 se llega a unos 13.9 GPa
- Para ir más allá: GroupKFold con la fórmula como grupo, para que los polimorfos no queden repartidos

Términos **Hiperparámetro:** ajuste del modelo que no se aprende de los datos (n.º de árboles) · **GroupKFold:** validación cruzada que mantiene juntos los materiales de un mismo grupo · **Polimorfos:** misma fórmula, distinta estructura cristalina (diamante y grafito)

02 • Paso 5: datos de Materials Project

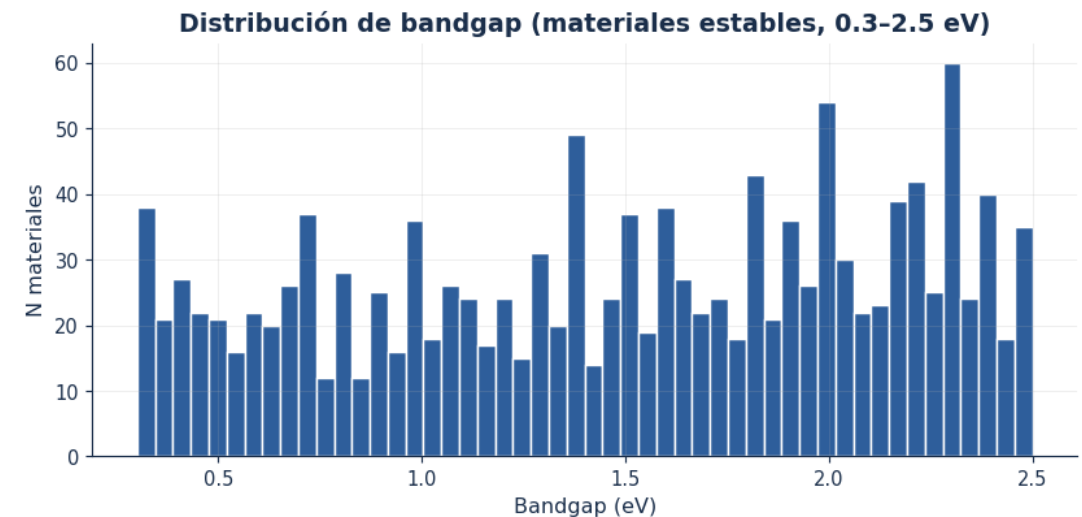
Celdas [10] y [11]

```
CACHE_FILE = DATOS / "mp_bandgap_cache.json"
df_mp = pd.read_json(CACHE_FILE) # caché local
# con API key (.env) se consulta en vivo:
# mpr.materials.summary.search(band_gap=(0.3, 2.5), ...)
```

Deben ver: 1364 materiales estables con gap entre 0.3 y 2.5 eV.

La API key: va en un archivo .env, nunca escrita en el notebook.

Fíjese en el filtro: solo semiconductores. Ningún metal (como el WC) está en este caché. Lo usamos en el paso 6.



Términos **API:** interfaz para consultar una base de datos desde un programa • **.env:** archivo local que guarda claves fuera del código • **Band gap:** brecha de energía entre bandas ocupadas y vacías; cero en metales • **eV:** electronvoltio, unidad de energía de los electrones • **Semiconductor:** material con gap pequeño, cuya conducción se controla

02 • Paso 6: diseño inverso por screening

Celdas [12] a [14]

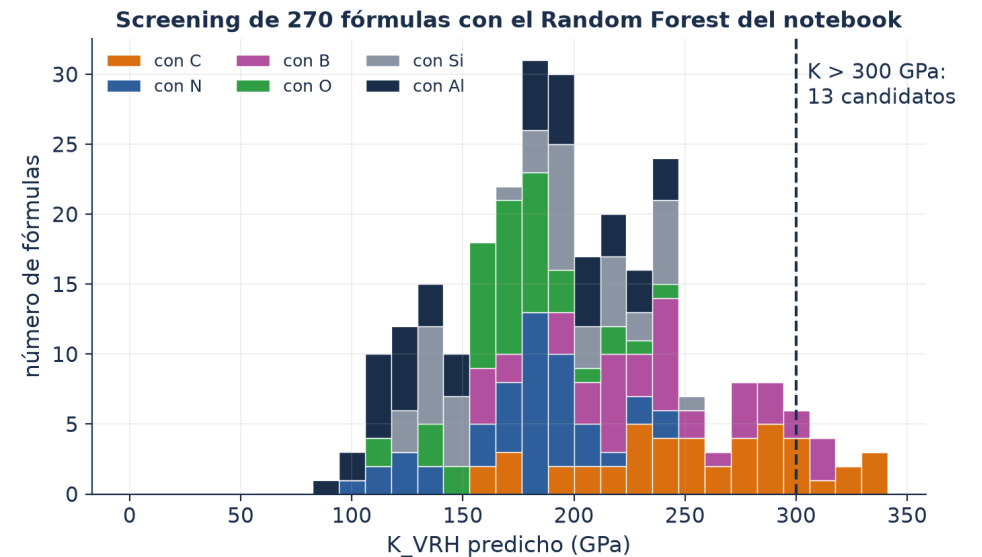
```
for A in ["Ti", "Zr", "Hf", "V", "Nb", "Ta", "Cr", "Mo", "W"]:  
    for B in ["C", "N", "B", "O", "Si", "Al"]: ... # AxBy  
X_cand_sc = scaler.transform(X_cand) # el MISMO escalador  
df_cand["K_VRH_pred"] = rf.predict(X_cand_sc)  
df_cand[df_cand["K_VRH_pred"] > 300]
```

Deben ver: 324 fórmulas, 270 únicas; 13 con K predicho > 300 GPa (W_2C , MoC, W_3C_2 , WC, BW...).

Validar la novedad: W_2C , MoC y WC ya estaban en el entrenamiento. Y en_MP sale False para todos porque el caché del paso 5 solo tiene semiconductores: compare siempre contra la base completa.

El techo: ninguno pasa de 331 GPa; un Random Forest no extrapola por encima de lo que vio.

Términos **Novedad:** que un compuesto no exista ya en las bases de referencia



Reto 2 del notebook 02 (10 min)



Celda [15]

- Amplíe el pool a ternarios A-B-C con un tercer conjunto de elementos
- Vuelva a calcular descriptores y a predecir
- ¿Aparece algún ternario con $K > 300$ GPa?



La pregunta importante

- ¿Confía más o menos en esas predicciones que en las de los binarios? ¿Por qué?
- Pista: ¿qué vio el modelo durante el entrenamiento?

02 • Paso 7: SHAP, ¿por qué predice lo que predice?

Celdas [16] y [17]

```
explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_test_sc) # (237, 132)
shap.summary_plot(shap_values, X_test_sc, ...)
```

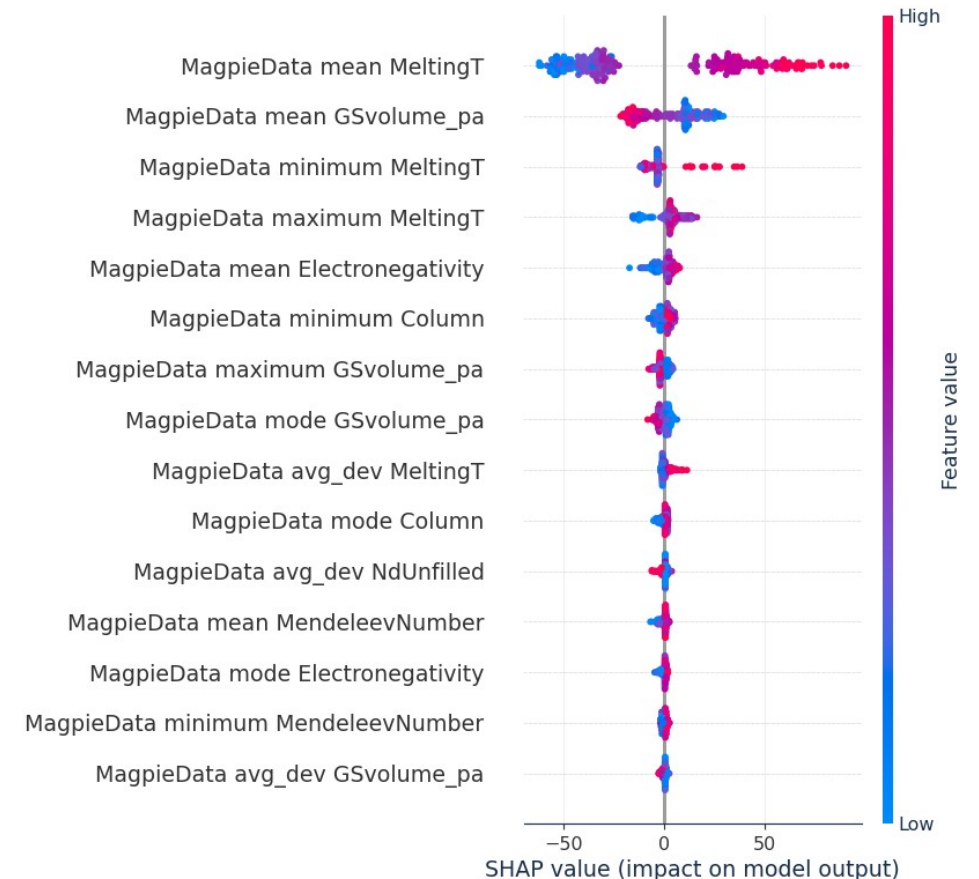
Cada punto es un material; cada fila, un descriptor.

Posición: cuántos GPa empujó la predicción hacia arriba o hacia abajo.

Color: el valor del descriptor (rojo alto, azul bajo).

Deben ver: temperatura de fusión media alta (rojo) empuja K hacia arriba.

Recuerde: SHAP dice qué usó el modelo, no qué causa la propiedad.



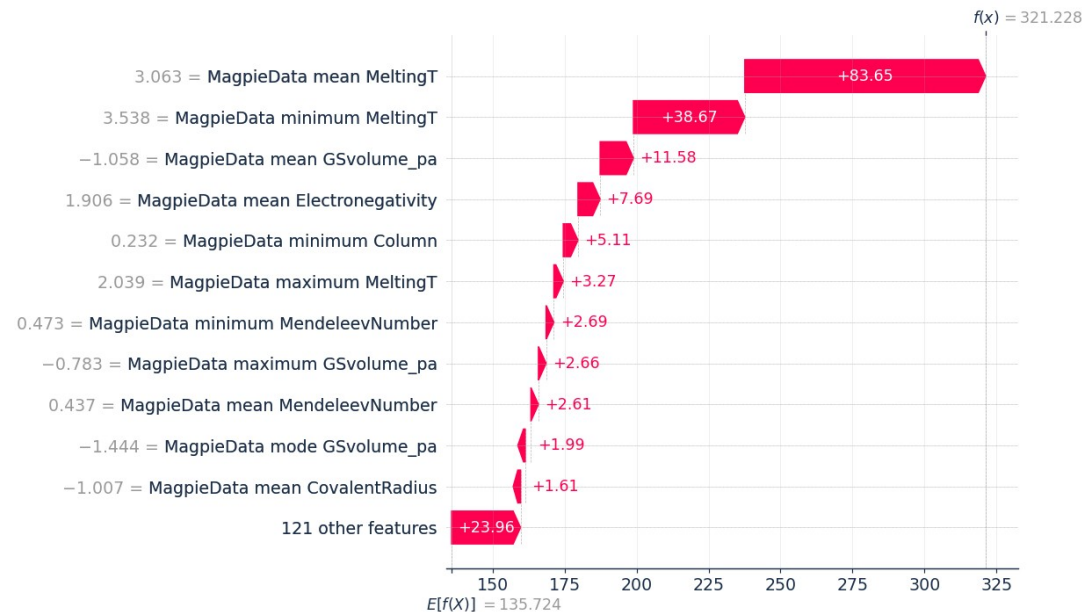
02 • Paso 8: explicar un solo material

Celda [18]

Waterfall: parte del promedio (136 GPa) y suma lo que aporta cada descriptor hasta llegar a la predicción.

Deben ver: el más rígido del test es WC: real 385 GPa, predicho 321 GPa. La temperatura de fusión suma +84 GPa.

Pregunte: ¿por qué se queda corto? Es el techo del Random Forest: ningún árbol vio valores mucho más altos.



Términos **Beeswarm y waterfall:** gráficos SHAP de todos los materiales (enjambre) o de uno solo (cascada)

Ejercicio del notebook 02 para casa

- En el caché de Materials Project, los materiales con gap entre 1.0 y 2.0 eV: ¿cuántos son? Haga un gráfico de band_gap contra E_f .
- Entrene un Random Forest que prediga band_gap desde descriptores Magpie (StrToComposition + ElementProperty). Reporte MAE y R^2 .
- ¿Cuál es el descriptor más importante? ¿Tiene sentido físico?

Idea clave Es el mismo flujo de los pasos 2 a 4, cambiando el dataset y la propiedad. Se revisa el miércoles a las 08:00.

El mismo método sirve para otros materiales

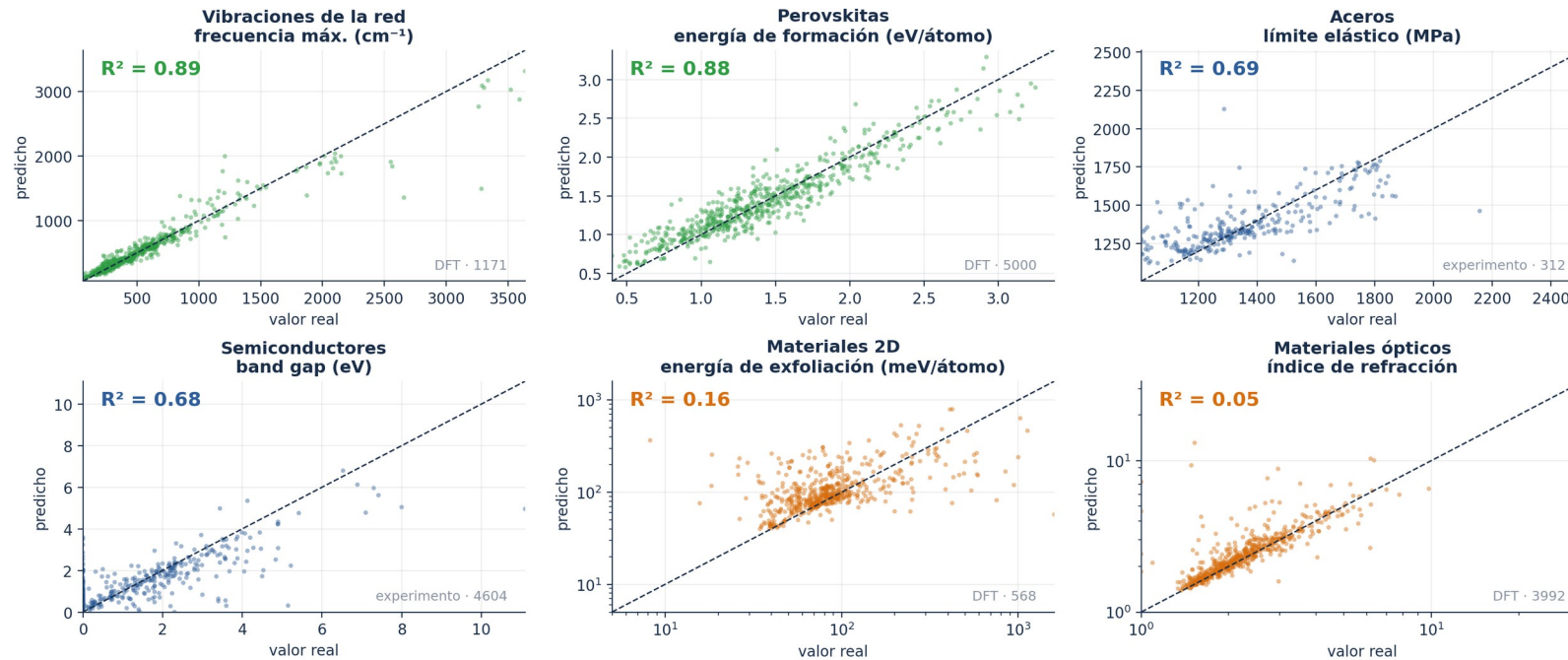
Datasets listos en matminer: se cargan con una línea, igual que el de hoy

Nombre en matminer	Materiales	Qué se predice, en sencillo	Origen · filas
elastic_tensor_2015 (hoy)	Cristales inorgánicos	Qué tan difícil es comprimirlo (K, en GPa).	DFT · 1181
matbench_steels	Aceros (Fe con C, Cr, Ni, Mo...)	Cuánta fuerza aguanta antes de quedar doblado para siempre (límite elástico, MPa).	Experimento · 312
matbench_expt_gap	Semiconductores y aislantes	El band gap: la energía mínima para que conduzca. Decide el color y si sirve para celdas solares.	Experimento · 4604
matbench_glass	Aleaciones metálicas	¿Se puede volver vidrio metálico (sólido sin orden cristalino)? Sí o no.	Experimento · 5680
matbench_perovskites	Perovskitas ABX ₃	Qué tan estable es el compuesto (energía de formación).	DFT · 18 928
matbench_phonons	Cristales	La vibración más rápida de los átomos. Átomos livianos y enlaces fuertes vibran más rápido.	DFT · 1265
matbench_jdft2d	Materiales en capas (tipo grafeno)	Cuánta energía cuesta despegar una capa.	DFT · 636
matbench_dielectric	Materiales ópticos	El índice de refracción: cuánto frena y desvía la luz.	DFT · 4764

Idea clave Para usarlos se cambian dos cosas: el nombre en load_dataset y la columna que se predice. El resto del notebook 02 queda igual.

Términos **Perovskita**: estructura tipo ABO₃, como SrTiO₃ · **Energía de formación**: energía del compuesto menos la de sus elementos, por átomo

El mismo código del notebook 02 en seis familias de materiales



Cada punto es un material: valor real contra predicho. Magpie + Random Forest de 200 árboles, validación cruzada de 5 partes, corrido en Kabré.

Idea clave Funciona bien cuando la propiedad depende sobre todo de qué elementos hay. Falla cuando depende de cómo están acomodados.

¿Por qué funciona en unos y en otros no?

El error típico del modelo comparado con adivinar siempre el promedio

Materiales	R ²	Error típico del modelo	Adivinando el promedio	Por qué
Vibraciones	0.89	72 cm ⁻¹	327 cm ⁻¹	Depende de qué tan livianos son los átomos y qué tan fuerte se enlazan: eso está en la composición.
Perovskitas	0.88	0.15 eV/átomo	0.42 eV/átomo	Misma estructura para todas; solo cambian los elementos. Caso ideal para Magpie.
Aceros	0.69	111 MPa	229 MPa	La misma receta con otro tratamiento térmico da otro acero. El proceso no está en la fórmula.
Semiconductores	0.68	0.45 eV	1.14 eV	Más de la mitad son metales (gap 0) y a varios el modelo les inventa un gap.
Vidrios metálicos	89 % de acierto		71 % (decir siempre «sí»)	Es clasificación (sí o no), no un número. Mejora, pero menos de lo que parece.
Materiales 2D	0.16	56 meV/átomo	71 meV/átomo	Lo que importa es cómo se apilan las capas, y Magpie no ve la estructura.
Índice de refracción	0.05	0.47	0.86	La mayoría cae cerca, pero unos pocos casos extremos (n hasta 62) hunden el R ² .

Idea clave Antes de confiar en un modelo pregunte: ¿esta propiedad depende solo de qué elementos hay? Si no, faltan descriptores de la estructura o del proceso.

Términos **Clasificación:** predecir una categoría (por ejemplo el sistema cristalino)

Pruébalo: el notebook 02 con otro dataset

Una celda nueva al final del notebook 02. Solo cambian las dos líneas marcadas

```
from matminer.datasets import load_dataset
from matminer.featurizers.conversions import StrToComposition
from matminer.featurizers.composition import ElementProperty
from sklearn.model_selection import KFold
nombre, objetivo = "matbench_steels", "yield strength" # <- cambiar aquí
df2 = load_dataset(nombre, data_home=str(DATOS / "matminer_cache"))
df2 = StrToComposition(target_col_id="comp").featurize_dataframe(df2, "composition")
ep = ElementProperty.from_preset("magpie"); ep.set_n_jobs(4)
df2 = ep.featurize_dataframe(df2, col_id="comp", ignore_errors=True)
fc = [c for c in df2.columns if "MagpieData" in c]
df2 = df2.dropna(subset=fc)
rf2 = RandomForestRegressor(n_estimators=200, n_jobs=4, random_state=42)
cv = KFold(5, shuffle=True, random_state=42)
print(cross_val_score(rf2, df2[fc], df2[objetivo], cv=cv, scoring="r2").mean())
```

Deben ver: con los aceros, $R^2 \approx 0.69$ en unos segundos. Con matbench_expt_gap y "gap expt", $R^2 \approx 0.68$ (tarda alrededor de un minuto).

data_home: el dataset se baja de internet a datos/matminer_cache (los nodos de Kabré sí tienen salida).

Si el dataset trae structure y no composition: `df2["comp"] = df2["structure"].apply(lambda s: s.composition)` en lugar de StrToComposition.

¿Y para prótesis y aplicaciones médicas?

Datasets abiertos para empezar un proyecto (todavía no probados en el curso)

Dataset	Qué tiene	Para qué sirve	Acceso
Aleaciones de Ti biomédicas (Ahmad et al., 2026)	689 aleaciones de Ti con Nb, Zr, Ta, Mo, Sn... Módulo de Young de 42 a 107 GPa y la fase (α , β)	El mejor para empezar: mismo flujo del notebook 02. Meta: bajar el módulo hacia el del hueso	CC BY 4.0 · NIMS MDR y Zenodo
Aleaciones de Ti biocompatibles (Manojlović y Marković, 2023)	238 aleaciones hechas casi solo con elementos biocompatibles	Igual que la anterior, más pequeña y enfocada	Materials Data Facility
MPEA (Borg et al., 2020)	630 aleaciones de varios elementos: dureza, resistencia, elongación, módulo	Aleaciones tipo TiNbZrTaHf que se estudian para implantes	GitHub de Citrine · Apache 2.0
Polímeros: Open Polymer Challenge, PolyInfo, degradación (Yuan et al., 2023)	Unos 11 000 polímeros con Tg y densidad (simulados); bases de propiedades medidas; degradación	Encajes y liners de prótesis, suturas. Ojo: vienen como SMILES, no sirve Magpie	Abiertos; PolyInfo pide registro
SciGlass	Más de 420 000 vidrios con propiedades físicas	Vidrios bioactivos para relleno óseo (no trae bioactividad)	GitHub · licencia MIT
Magnesio biodegradable (Suh et al., 2023)	Estudios con unas 840 mediciones de corrosión y resistencia	Implantes que se disuelven solos	No encontré los datos abiertos

Idea clave Hueso: unos 10 a 30 GPa. Ti-6Al-4V: unos 110 GPa. Un implante muy rígido le quita la carga al hueso y este se debilita. Ojo: casi no hay datos abiertos de respuesta biológica.

Términos **SMILES**: notación de texto para moléculas (PSMILES y BigSMILES para polímeros)

Fuente: módulos del hueso y del Ti-6Al-4V: Niinomi 2008. Referencias completas de cada dataset al final.

Referencias (1/4)

- [1] de Jong, M., Chen, W., et al. (2015). **Charting the complete elastic properties of inorganic crystalline compounds.** *Scientific Data* 2, 150009. doi:10.1038/sdata.2015.9
- [2] Ong, S. P., Richards, W. D., Jain, A., et al. (2013). **Python Materials Genomics (pymatgen): A robust, open-source python library for materials analysis.** *Comput. Mater. Sci.* 68, 314–319. doi:10.1016/j.commatsci.2012.10.028
- [3] Ward, L., Agrawal, A., Choudhary, A., Wolverton, C. (2016). **A general-purpose machine learning framework for predicting properties of inorganic materials.** *npj Comput. Mater.* 2, 16028. doi:10.1038/npjcompumats.2016.28
- [4] Ward, L., Dunn, A., Faghaninia, A., et al. (2018). **Matminer: An open source toolkit for materials data mining.** *Comput. Mater. Sci.* 152, 60–69. doi:10.1016/j.commatsci.2018.05.018
- [5] Jain, A., Ong, S. P., Hautier, G., et al. (2013). **The Materials Project: A materials genome approach to accelerating materials innovation.** *APL Materials* 1, 011002. doi:10.1063/1.4812323
- [6] Breiman, L. (2001). **Random forests.** *Machine Learning* 45, 5–32. doi:10.1023/A:1010933404324

Referencias (2/4)

- [7] Lundberg, S. M., Lee, S.-I. (2017). **A unified approach to interpreting model predictions.** *NeurIPS* 30. [arXiv:1705.07874](https://arxiv.org/abs/1705.07874)
- [8] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). **Scikit-learn: Machine learning in Python.** *J. Mach. Learn. Res.* 12, 2825–2830
- [9] Dunn, A., Wang, Q., Ganose, A., Dopp, D., Jain, A. (2020). **Benchmarking materials property prediction methods: the Matbench test set and Automatminer reference algorithm.** *npj Comput. Mater.* 6, 138. [doi:10.1038/s41524-020-00406-3](https://doi.org/10.1038/s41524-020-00406-3)
- [10] Zhuo, Y., Mansouri Tehrani, A., Brgoch, J. (2018). **Predicting the band gaps of inorganic solids by machine learning.** *J. Phys. Chem. Lett.* 9, 1668–1673. [doi:10.1021/acs.jpclett.8b00124](https://doi.org/10.1021/acs.jpclett.8b00124)
- [11] Niinomi, M. (2008). **Mechanical biocompatibilities of titanium alloys for biomedical applications.** *J. Mech. Behav. Biomed. Mater.* 1, 30–42. [doi:10.1016/j.jmbbm.2007.07.001](https://doi.org/10.1016/j.jmbbm.2007.07.001)
- [12] Ahmad, H., Haider, M., Iqbal, Z., Zarif, M., Hassan, S. (2026). **Machine-learning prediction of Young's modulus in multi-component titanium-based biomedical alloys using extended thermodynamic descriptors.** *Sci. Technol. Adv. Mater.: Methods* 6, 2691685. [doi:10.1080/27660400.2026.2691685](https://doi.org/10.1080/27660400.2026.2691685)

Referencias (3/4)

- [13] Manojlović, V., Marković, G. (2023). **Titanium alloys database for medical applications**. *Metall. Mater. Data* 1, 1–6 (datos: Materials Data Facility, doi:10.18126/rwuh-osea). doi:10.30544/MMD5
- [14] Borg, C. K. H., Frey, C., Moh, J., Pollock, T. M., Gorsse, S., Miracle, D. B., et al. (2020). **Expanded dataset of mechanical properties and observed phases of multi-principal element alloys**. *Scientific Data* 7, 430. doi:10.1038/s41597-020-00768-9
- [15] Liu, G., Alosious, S., Mahajan, S., Inae, E., Zhu, Y., et al. (2025). **Open Polymer Challenge: Post-Competition Report**. arXiv:2512.08896
- [16] Otsuka, S., Kuwajima, I., Hosoya, J., Xu, Y., Yamazaki, M. (2011). **PoLyInfo: Polymer database for polymeric materials design**. *Int. Conf. Emerging Intelligent Data and Web Technologies*, 22–29. doi:10.1109/EIDWT.2011.13
- [17] Yuan, W., Hibi, Y., Tamura, R., Sumita, M., Nakamura, Y., Naito, M., et al. (2023). **Revealing factors influencing polymer degradation with rank-based machine learning**. *Patterns* 4, 100846. doi:10.1016/j.patter.2023.100846
- [18] EPAM Systems. **SciGlass: base de datos de propiedades de vidrios (más de 420 000 composiciones)**. github.com/epam/SciGlass

Referencias (4/4)

[19] Han, T., Stone-Weiss, N., Huang, J., Goel, A., Kumar, A. (2020). **Machine learning as a tool to design glasses with controlled dissolution for healthcare applications.** *Acta Biomater.* 107, 286–298. doi:10.1016/j.actbio.2020.02.037

[20] Suh, J., Suh, B., Bae, J., Kim, Y. (2023). **Machine learning-based design of biodegradable Mg alloys for load-bearing implants.** *Mater. Des.* 225, 111442. doi:10.1016/j.matdes.2022.111442



¡Gracias!

Tomás Rojas Solórzano
Universidad de Costa Rica · tomas.rojas_s@ucr.ac.cr